```
#include <iostream>
Using namespace std;

Struct Node {
   Int data;
   Node *left, *right;
   Bool lthread, rthread;
};

Node* createNode(int x) {
   Node* p = new Node;
   p->data = x;
   p->left = NULL;
   p->right = NULL;
   p->lthread = true;
   p->rthread = true;
   return p;
}

Node* insert(Node* root, int x) {
   Node* p = root;
   Node* parent = NULL;

   While (p != NULL) {
      If (x == p->data)
         Return root;
```

```
    Parent = p;

    If (x < p->data) {
      If (p->lthread == false)
        P = p->left;
      Else
        Break;
    } else {
      If (p->rthread == false)
        P = p->right;
      Else
        Break;
    }
}

Node* temp = createNode(x);

If (parent == NULL) {
    Root = temp;
} else if (x < parent->data) {
    Temp->left = parent->left;
    Temp->right = parent;
    Parent->left = temp;
    Parent->lthread = false;
} else {
```

```
      Temp->left = parent;
      Temp->right = parent->right;
      Parent->right = temp;
      Parent->rthread = false;
   }

   Return root;
}

Node* findPred(Node* p) {
   If (p->lthread)
      Return p->left;

   P = p->left;

   While (p->rthread == false)
      P = p->right;

   Return p;
}

Node* findSucc(Node* p) {
   If (p->rthread)
      Return p->right;

   P = p->right;
```

```
  While (p->lthread == false)
    P = p->left;

  Return p;
}

Node* search(Node* root, int x) {
  Node* p = root;

  While (p != NULL) {
    If (p->data == x)
      Return p;

    If (x < p->data) {
      If (p->lthread)
        Return NULL;
      P = p->left;
    } else {
      If (p->rthread)
        Return NULL;
      P = p->right;
    }
  }

  Return NULL;
```

```
}

Void showInorder(Node* root, int a[], int &n) {
   Node* p = root;

   If (p == NULL)
      Return;

   While (p->lthread == false)
      P = p->left;

   While (p != NULL) {
      A[n++] = p->data;

      If (p->rthread)
         P = p->right;
      Else {
         P = p->right;

         While (p->lthread == false)
            P = p->left;
      }
   }
}

Int main() {
```

```
Node* root = NULL;
Int n, x;
Int a[100], count = 0;
Bool correct = true;

Cout << “Enter number of nodes: “;
Cin >> n;

If (n <= 0 || n > 100) {
   Cout << “Invalid number of nodes.”;
   Return 0;
}

Cout << “Enter node values: “;

For (int I = 0; I < n; i++) {
   Cin >> x;
   Root = insert(root, x);
}

showInorder(root, a, count);

cout << “\nInorder Traversal: “;
for (int I = 0; I < count; i++)
   cout << a[i] << “ “;
```

```
cout << “\n\nNode\tPredecessor\tSuccessor\n”;

for (int I = 0; I < count; i++) {
   Node* p = search(root, a[i]);
   Node* pre = findPred(p);
   Node* suc = findSucc(p);

   Cout << a[i] << “\t”;

   If (pre == NULL)
      Cout << “Not Exists”;
   Else
      Cout << pre->data;

   Cout << “\t\t”;

   If (suc == NULL)
      Cout << “Not Exists”;
   Else
      Cout << suc->data;

   Cout << “\n”;

   If (I == 0) {
      If (pre != NULL)
         Correct = false;
```

```
    } else {
      If (pre == NULL || pre->data != a[I – 1])
        Correct = false;
    }

    If (I == count – 1) {
      If (suc != NULL)
        Correct = false;
    } else {
      If (suc == NULL || suc->data != a[I + 1])
        Correct = false;
    }
  }

  Cout << “\nVerification: “;

  If (correct)
    Cout << “Results match with inorder traversal.”;
  Else
    Cout << “Results do not match.”;

  Return 0;
}
```